

STUDY NOTES — BRIEF

Bidaw 학습 노트 (간략판)

3-4 페이지 다이제스트

Bidaw — FAST '26 paper seminar 보조자료

Bidaw: Enhancing Key-Value Caching for Interactive LLM Serving via Bidirectional Computation-
Storage Awareness

Hu et al., FAST '26

1. 한 줄 요약

Compute engine과 two-tier storage(host DRAM + SSD)가 **양방향**으로 정보를 흘려 KV 로딩 병목을 해소. Compute → Storage : **previous answer length** · Storage → Compute : **KV location & size**.

2. 배경 (꼭 필요한 만큼만)

- **KV cache** = attention의 K, V tensor를 보존해 재계산을 피하는 자료구조. 토큰 수에 **선형**으로 커진다.
- **Two-tier**: perf layer = host DRAM, capacity layer = SSD. 쓰기는 백그라운드, 읽기는 **critical path**.
- **MHA vs GQA**: MHA는 head별 K,V 분리 → KV가 큼. GQA는 query head들이 K,V 공유 → KV 자체가 작음. 이 차이가 §6의 적용 범위 결정.
- **HRRN** (1971, Brinch Hansen): $R = 1 + \text{waiting}/\text{service}$. 짧은 작업 우대 + 기아 방지의 고전 절충안.
- **Belady** (1966): 미래에 가장 늦게 다시 쓰일 페이지를 evict — 모든 캐시 정책의 **오라클 상한**. 미래를 알아야 해서 보통 측정용.

3. 문제 — 세 관찰

#	무엇	수치
1	동시 캐시 KV 양 $\gg$ perf layer	30 users/min, OPT-13B → 480 GB (perf 200 GB)
2	약한 temporal locality	80% access > perf layer 크기, hit rate $\approx$ 20 %
3	KV loading time 분산 큼	CV > 90 % (5초 윈도우 안에서도)

원인 한 줄: **compute와 storage가 서로 unaware**. - Compute가 storage I/O 길이 모름 → FCFS dispatch가 head-of-line blocking - Storage가 사용자 대화 패턴 모름 → eviction이 자기 측 history만 사용

4. 핵심 아이디어 — Bidirectional awareness

방향	신호	어디 쓰이나
Storage → Compute	KV의 위치(layer)와 크기	Mechanism 1 (scheduling)
Compute → Storage	직전 라운드의 답변 길이	Mechanism 2 (eviction)

5. 세 가지 메커니즘

5.1 Mechanism 1 — I/O-aware Scheduling

- **Dual queue**: ready (KV가 perf layer) / preparing (KV가 capacity layer).
- **disk-HRRN**으로 preparing queue 우선순위: $R = 1 + \text{waiting_time} / \text{KV_size}$ 작은 KV 먼저 promote, 큰 KV는 waiting이 늘어 결국 따라잡음(starvation 방지).
- Promote 시 ready queue 안 위치는 **원래 도착 시각** 기준 (tail latency 폭주 방지).
- **결과**: 평균 큐잉 시간 5.76 → 2.45 s (-57.5 %), 단독 throughput +1.58×.

5.2 Mechanism 2 — Previous-answer-based Eviction

- 핵심 발견: 이전 답변 길이 ↔ 다음 KV access의 **reuse distance 하한** — Spearman $\rho = 0.94 \sim 0.98$.
- 직관: 답변 길다 → 사용자가 오래 읽음 → 그 사이 다른 사용자 KV가 더 많이 끼어듦.
- **그러나 거리 ≠ miss**: reuse distance가 promising 영역($\approx 200 \sim 440$ GB)이면 Belady는 hit $\approx 100\%$, LRU는 0~40 %. 단순 거리 기반 evict는 promising을 학살함.
- **알고리즘**: 1. Promising 영역을 **m개 fine-grained bucket**으로 분할. 2. 각 bucket의 hit rate를 ghost cache(metadata만 보유) 위 Belady 시뮬레이션으로 측정. 3. 사용자별 과거 분포로 다음 access의 bucket 확률 추정. 4. **답변 길이로 reuse distance 하한** → 더 작은 bucket 확률을 0으로 truncate. 5. **Equation 2** 로 **Overall_potential** 계산: $\text{Overall_potential} = \text{prob_small} \cdot 1.0 + \text{prob_extreme} \cdot 0.0 + \sum_i \text{prob_promising}(i) \cdot \text{hit_promising}(i)$ 6. 가장 낮은 potential을 가진 KV를 evict.
- **결과**: miss rate -57.6 % (vs queue-enhanced) / -69.9 % (vs FIFO/LRU). 단독 throughput +1.25×.

5.3 Mechanism 3 — Storage-Efficient Tensor Caching

- KV tensor 대신 **Tensor 6** (정규화된 activation, FFN 직전)을 캐시.
- **Cost efficiency** = saved compute / required space (GFLOPs/MB).
- Tensor 6: **51 GFLOPs/MB**, KV tensor: 30.5 → 1.67× 효율.
- 변환은 GPU 1 step, low-priority CUDA stream에서 처리(< 100 ms).

- **MHA-only**. GQA는 KV 자체가 작아 변환 비용이 우위 잡아먹음 → 그냥 KV 캐시.
- 단독 throughput +1.10×.

6. 평가 한눈에

지표	결과
평균 latency 단축	최대 3.58 ×
Throughput 향상	1.43 – 1.83 × (동일 latency 기준)
Tail (P90 / P95 / P99)	-53 / -49 / -47 % vs CachedAttention
Eviction miss rate	-69.9 % vs FIFO/LRU
Memory sensitivity	120 GB host에서도 baseline 200 GB 수준

Ablation: vanilla → +sched 1.58× → +evict 1.97× → +tensor 2.17×

7. 한계 4가지 (꼭 마지막에 짚을 것)

1. Single-user KV 재사용만 — cross-user 공유(MeanCache류)는 직교 영역.
2. GQA에선 Mechanism 3 비활성.
3. ShareGPT처럼 timestamp 부재 trace에선 eviction 효과 약화.
4. Single GPU 노드 평가 — disaggregated/multi-node 미평가.

8. 자기 점검 5문제 (입으로 답할 수 있어야 한다)

1. Weighted reuse distance를 한 문장으로 정의하라.
2. 답변 길이가 reuse distance와 양의 상관인 이유는?
3. disk-HRRN 식 한 줄. 원래 HRRN과 다른 점은?
4. Belady가 ghost cache에서 왜 가능한가?
5. GQA에서 Mechanism 3을 비활성화하는 이유는?